

Advanced Kubernetes Task for Library Management System with Database Integration and GitHub Skills Testing

Objective:

Deploy the Library Management System and its MySQL database on a Kubernetes cluster using advanced Kubernetes features, test GitHub skills, and implement CI/CD pipelines. This task involves setting up the application, managing deployments, advanced networking, persistent storage, autoscaling, CI/CD integration, and GitHub version control.

Task Overview:

- GitHub Repository Setup: Create a GitHub repository and manage branches, issues, and pull requests.
- Containerization of the Application: Dockerize the Library Management System, build and push the Docker image.
- Database Setup in Kubernetes: Deploy MySQL with persistent storage in Kubernetes.
- Library Management System Deployment: Deploy the application using Kubernetes resources and implement blue-green deployment.
- Kubernetes Configurations: Use ConfigMaps, Secrets, and implement horizontal pod autoscaling.
- Advanced Networking and Security: Define network policies and implement TLS/SSL termination.
- CI/CD and GitOps Integration: Set up ArgoCD for GitOps and implement CI/CD pipelines with GitHub Actions.

Detailed Instructions:

Step 1: GitHub Repository Setup

1. Create a GitHub Repository: Create a new GitHub repository named `library-management-system`.
2. Branching Strategy: Implement a branching strategy with `main`, `develop`, and `feature/*` branches.

3. GitHub Issues and Pull Requests: Use GitHub Issues to track tasks and Pull Requests (PRs) to manage code changes.

Step 2: Containerize the Library Management System

1. Create a Dockerfile: Create a `Dockerfile` in the root directory of the Library Management System.
2. Build Docker Image: Build the Docker image using the Dockerfile.
3. Push Docker Image: Push the Docker image to Docker Hub.
4. Commit Changes: Commit the Dockerfile and related files to the GitHub repository.

Step 3: Database Setup in Kubernetes

1. Create a Persistent Volume for MySQL: Define a Persistent Volume (PV) and Persistent Volume Claim (PVC) to persist MySQL data.
2. Create a MySQL Deployment: Define and apply a Deployment YAML file for MySQL.
3. Create a MySQL Service: Define and apply a Service YAML file to expose the MySQL database.
4. Commit Changes: Commit the MySQL Kubernetes manifests to the GitHub repository.

Step 4: Library Management System Deployment

1. Create a Kubernetes Deployment: Define and apply a Deployment YAML file for the Library Management System
2. Commit Changes: Commit the Kubernetes manifests for the Library Management System to the GitHub repository.

Step 5: Kubernetes Configurations

1. ConfigMaps and Secrets: Create ConfigMaps and Secrets for application configuration and sensitive data.
2. Horizontal Pod Autoscaler: Implement an autoscaler to manage scaling based on CPU/memory usage.
3. Commit Changes: Commit the ConfigMap and Secret YAML files to the GitHub repository.

Step 6: CI/CD and GitOps Integration

1. GitOps with ArgoCD: Set up ArgoCD for GitOps-style continuous deployment, syncing Kubernetes manifests from a Git repository.
2. CI Pipeline with GitHub Actions: Implement a GitHub Actions workflow to automate testing and deployment.